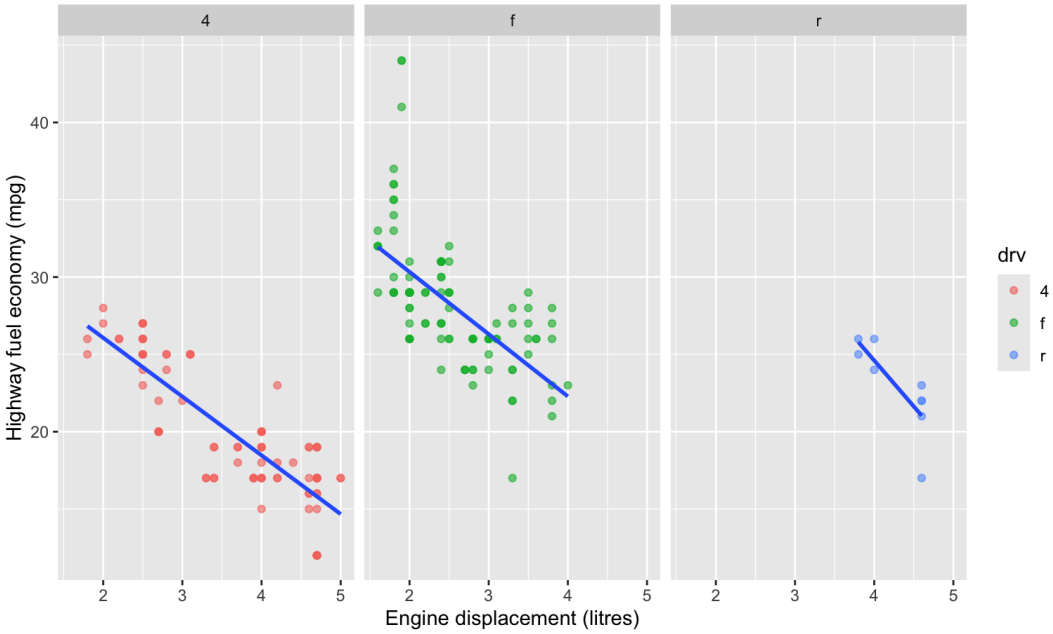
Problem #1

Part A

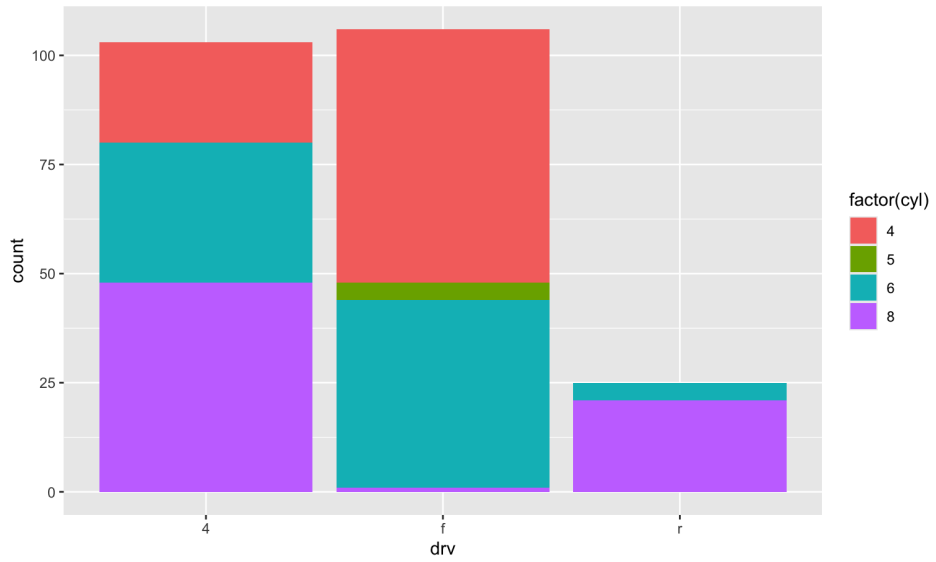
```
p_final <- vehicles |>
  filter(displ <= 5) |>
  ggplot(aes(x = displ, y = hwy)) +
  geom_point(aes(color = drv), alpha = 0.6) +
  geom_smooth(method = "lm", formula = y ~ x, se = FALSE) +
  facet_wrap(~ drv) +
  labs(x = "Engine displacement (litres)",
       y = "Highway fuel economy (mpg)")
p_final
```



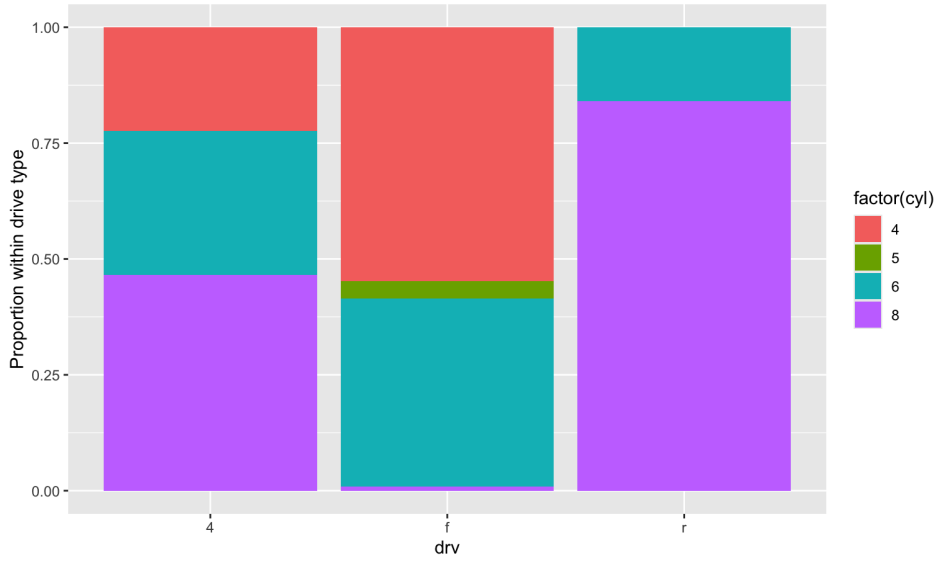
There are three fitted lines as three different panels are created for each drive type, all with their own data, even though drv is mapped to color only in the point layer.

Part B

```
p_positions <- ggplot(vehicles,
                      aes(x = drv, fill = factor(cyl)))
p_stack <- p_positions + geom_bar(position = "stack")
p_stack
```



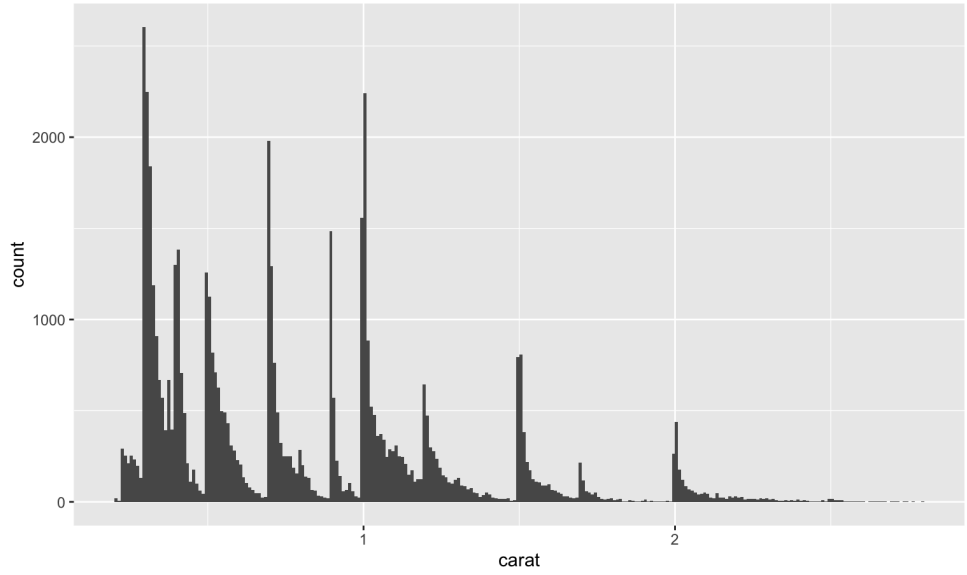

```
p_positions <- ggplot(vehicles,
                      aes(x = drv, fill = factor(cyl)))
p_fill <- p_positions + geom_bar(position = "fill") +
  labs(y = "Proportion within drive type")
p_fill
```



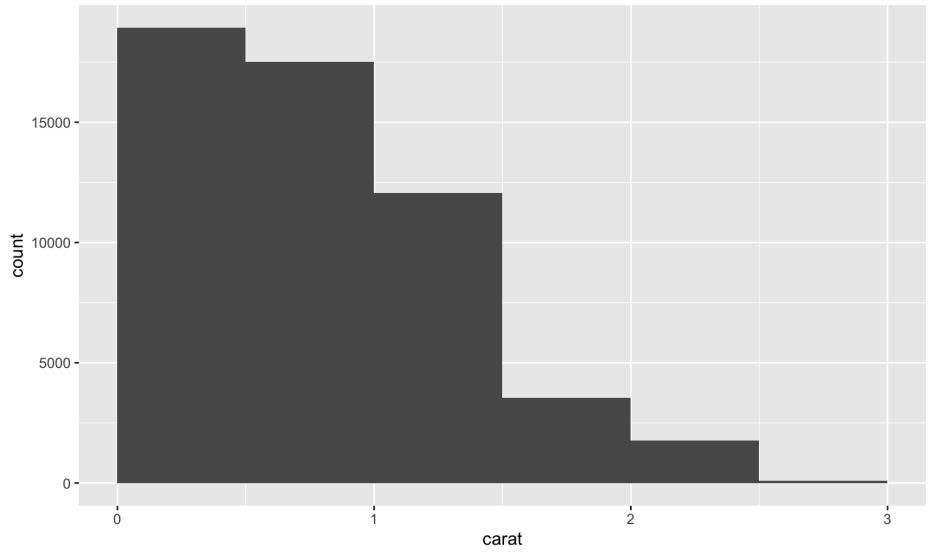
The heights in the second plot represent proportion, not sample sizes, so each has a height of 1. The plot compares the proportions of cylinder groups within each drive type instead of the total counts.

Part C

```
smaller <- gems |> filter(carat < 3)
p_fine <- ggplot(smaller, aes(x = carat))
p_fine + geom_histogram(binwidth = 0.01, boundary = 0, closed = "right")
```




```
p_coarse <- ggplot(smaller, aes(x = carat))
p_coarse + geom_histogram(binwidth = 0.5, boundary = 0, closed = "right")
```




```
gems |>
  filter(carat %in% c(0.99, 1.00)) |>
  count(carat)
```

The narrower bins allow the peaks and gaps within each bin to be easier seen. This allows for a better understanding and visual details of specific frequencies.

Problem #2

```
input_file <- "data_sources.csv"

file.copy(input_file, file.path(out_dir, basename(input_file)), overwrite = TRUE)

raw <- read_csv(input_file, show_col_types = FALSE)
n_raw <- nrow(raw)

coord_match <- str_match(as.character(raw$LOCATION),
                         "^\\s*\\(?\\s*(-?[0-9]+\\.?[0-9]*)\\s*,\\s*(-?[0-9]+\\.?[0-9]*)\\s*\\)?\\s*$")
dat <- raw %>%
  mutate(
    latitude = as.numeric(coord_match[, 2]),
    longitude = as.numeric(coord_match[, 3]),
    facility_name = str_squish(as.character(BRANCH))
  )

bad_parse <- is.na(dat$latitude) | is.na(dat$longitude)
bad_range <- !bad_parse &
  (dat$latitude < 41 | dat$latitude > 43 |
     dat$longitude < -89 | dat$longitude > -86
  )
bad_name <- is.na(dat$facility_name) | dat$facility_name == ""

removal_reason <- rep(NA_character_, nrow(dat))
removal_reason[bad_parse] <- "Missing or unparseable coordinates"
removal_reason[!bad_parse & bad_range] <- "Coordinates outside Chicago-area plausibility bounds"
removal_reason[!bad_parse & !bad_range & bad_name] <- "Missing facility name"

clean <- dat %>%
  mutate(removal_reason = removal_reason) %>%
  filter(is.na(removal_reason)) %>%
  mutate(site_id = sprintf("LIB%03d", row_number())) %>%
  select(site_id, facility_name, latitude, longitude)

duplicate_coordinate_rows <- sum(duplicated(clean[c("latitude", "longitude")]))
duplicate_name_rows <- sum(duplicated(clean$facility_name))

record_counts <- tibble(
  stage = c("Raw records", "After coordinate/name validation", "Retained sites"),
  count = c(n_raw, nrow(clean), nrow(clean))
)
```

```
data_checks <- tibble(
  check = c("Missing/unparseable coordinates",
            "Coordinates outside Chicago-area plausibility bounds",
            "Exact duplicate coordinate pairs",
            "Duplicate facility names",
            "Records removed"),
  count = c(sum(bad_parse), sum(bad_range),
            duplicate_coordinate_rows, duplicate_name_rows,
            sum(!is.na(removal_reason))),
  action = c("Removed if present", "Removed if present",
             "Retained; no coordinate-based deletion",
             "Retained; unique site IDs distinguish records",
             "See removal_log.csv")
)

data_sources <- tibble(
  source_file = basename(input_file),
  source_publisher = "Chicago Public Library (dataset identity inferred from branch records)",
  source_url = "Not provided in uploaded file",
  date_accessed = "2026-11-08",
  notes = "Raw source CSV preserved unchanged; LOCATION parsed as (latitude, longitude)."
)



sites_ll <- st_as_sf(clean, coords = c("longitude", "latitude"),
                     crs = 4326, remove = FALSE)
sites_m <- st_transform(sites_ll, 26916)



gpkg <- file.path(out_dir, "chicago_library_spatial.gpkg")
if (file.exists(gpkg)) file.remove(gpkg)
st_write(sites_m, gpkg, layer = "sites_m", quiet = TRUE)

buffer_500 <- st_union(st_buffer(sites_m, dist = 500, nQuadSegs = 64))
buffer_1000 <- st_union(st_buffer(sites_m, dist = 1000, nQuadSegs = 64))
buffer_500_sf <- st_sf(radius_m = 500, geometry = buffer_500)
buffer_1000_sf <- st_sf(radius_m = 1000, geometry = buffer_1000)
st_write(buffer_500_sf, gpkg, layer = "buffer_500m", quiet = TRUE)
st_write(buffer_1000_sf, gpkg, layer = "buffer_1000m", quiet = TRUE)

buffer_summary <- tibble(
  radius_m = c(500, 1000),
  union_area_km2 = c(as.numeric(st_area(buffer_500)) / 1e6,
```

```
                as.numeric(st_area(buffer_1000)) / 1e6
)

p_sites <- ggplot() +
  geom_sf(data = sites_m, aes(color = "Library facility"), size = 2.1, alpha = 0.9) +
  scale_color_manual(name = "Map layers", values = c("Library facility" = "#b23a48")) +
  labs(
    title = "Chicago Public Library Facilities",
    subtitle = paste(nrow(sites_m), "retained facility locations"),
    caption = "Source: uploaded Libraries – Locations, Contact Information, and Usual Hours of Operation
CSV | Date: 2026-11-08 | CRS: EPSG:26916"
  ) +
  theme_minimal() +
  theme(legend.position = "bottom")

p_buffers <- ggplot() +
  geom_sf(data = buffer_500_sf, aes(fill = "Combined 500 m buffer"),
          color = "#277da1", alpha = 0.55) +
  geom_sf(data = sites_m, aes(color = "Library facility"), size = 2.1) +
  scale_fill_manual(name = "Map layers", values = c("Combined 500 m buffer" = "#8ecae6")) +
  scale_color_manual(name = "Map layers", values = c("Library facility" = "#b23a48")) +
  labs(
    title = "Chicago Public Library Facilities and 500-Meter Buffer",
    subtitle = "Overlapping facility buffers dissolved; overlap counted once",
    caption = "Source: uploaded Libraries – Locations, Contact Information, and Usual Hours of Operation
CSV | Date: 2026-11-08 | Not clipped to a study boundary | CRS: EPSG:26916"
  ) +
  theme_minimal() +
  theme(legend.position = "bottom")

print(record_counts)

# A tibble: 3 × 2
  stage                       count
  <chr>                       <int>
1 Raw records                    82
2 After coordinate/name validation    82
3 Retained sites                 82

print(data_checks)

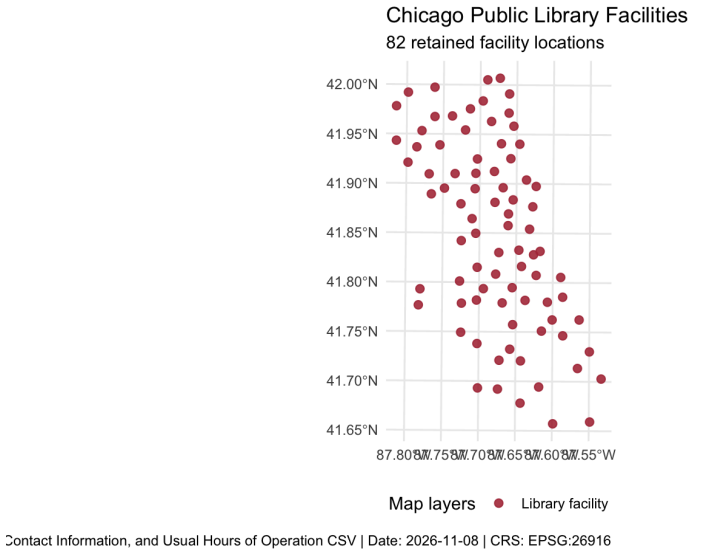
A tibble: 5 × 3
  check                                 count action
  <chr>                                 <int> <chr>
```

1 Missing/unparseable coordinates 0 Removed if present
2 Coordinates outside Chicago-area plausibility bounds 0 Removed if present
3 Exact duplicate coordinate pairs 0 Retained; no coordinate-based d…
4 Duplicate facility names 0 Retained; unique site IDs disti…
5 Records removed 0 See removal_log.csv

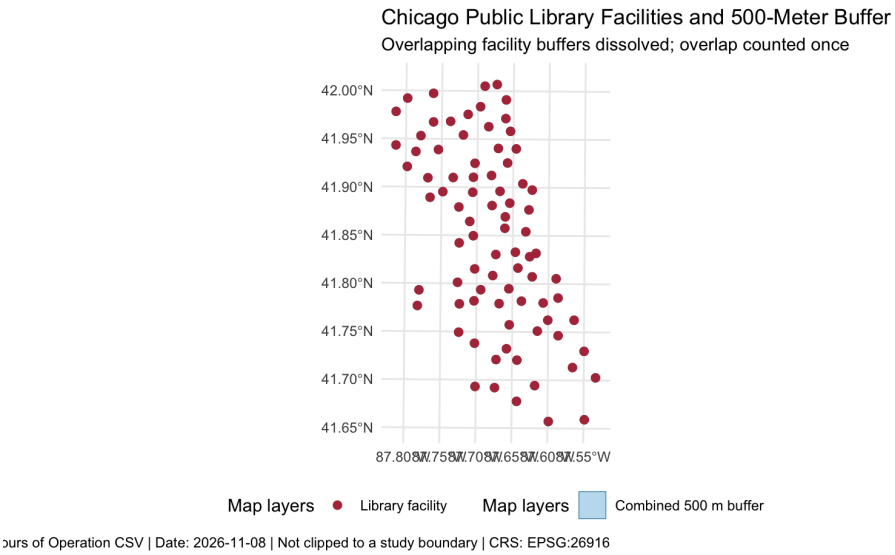
print(buffer_summary)

A tibble: 2 × 2
radius_m union_area_km2
<dbl> <dbl>
1 500 64.3
2 1000 250.

print(p_sites)



print(p_buffers)



Problem #3

```
sites_tab <- read_csv("Boundaries_Community_Areas.csv")

sites_tab <- read_csv("data_sources.csv", show_col_types = FALSE)
sites <- st_as_sf(sites_tab, coords = c("longitude", "latitude"),
                  crs = 4326, remove = FALSE) |>
  st_transform(26916)

boundary_tab <- read_csv("Boundaries_Community_Areas.csv",
                         show_col_types = FALSE)
boundary_tab <- boundary_tab |>
  mutate(district_id = sprintf("%02d", as.integer(AREA_NUMBE)),
         district_name = tools::toTitleCase(tolower(COMMUNITY)),
         geometry = st_as_sfc(the_geom, crs = 4326))
areas <- st_as_sf(boundary_tab, sf_column_name = "geometry") |>
  st_transform(26916)

areas$geometry_valid <- st_is_valid(areas)
areas$area_km2 <- as.numeric(st_area(areas)) / 1e6
boundary_checks <- tibble(
  check = c("boundary_records", "valid_nonempty_polygons",
            "unique_district_ids", "unique_district_names",
            "nonpositive_area_polygons"),
  result = c(nrow(areas),
             sum(areas$geometry_valid & !st_is_empty(areas)),
             n_distinct(areas$district_id),
             n_distinct(areas$district_name),
             sum(areas$area_km2 <= 0 | is.na(areas$area_km2))),
  note = c("Expected 77 community areas",
           "All polygons should be valid and non-empty",
           "Must equal number of areas",
           "Must equal number of areas",
           "Must be zero")
)

hits <- st_intersects(sites, areas)
site_match_audit <- tibble(
  site_id = sites$site_id,
  n_area_matches = lengths(hits)
)

match_summary <- tibble(
  match_category = c("Zero areas", "Exactly one area", "Multiple areas"),
```

```
  n_sites = c(sum(site_match_audit$n_area_matches == 0),
              sum(site_match_audit$n_area_matches == 1),
              sum(site_match_audit$n_area_matches > 1))
)

exactly_one <- which(lengths(hits) == 1)
assigned <- sites[exactly_one, ]
assigned$district_id <- vapply(hits[exactly_one],
                               function(x) areas$district_id[x[1]], character(1))
unassigned_or_ambiguous <- site_match_audit |>
  filter(n_area_matches != 1)
write_csv(unassigned_or_ambiguous,
          file.path(out, "sites_not_exactly_one_match.csv"))

site_counts <- assigned |>
  st_drop_geometry() |>
  count(district_id, name = "n_sites")
district_counts <- areas |>
  st_drop_geometry() |>
  select(district_id, district_name, area_km2) |>
  left_join(site_counts, by = "district_id") |>
  mutate(n_sites = coalesce(n_sites, 0L),
         sites_per_km2 = n_sites / area_km2) |>
  select(district_id, district_name, n_sites, area_km2, sites_per_km2) |>
  arrange(as.integer(district_id))

st_write(areas, file.path(out, "community_areas.gpkg"),
         layer = "community_areas", delete_layer = TRUE, quiet = TRUE)
st_write(assigned, file.path(out, "sites_assigned.gpkg"),
         layer = "sites_assigned", delete_layer = TRUE, quiet = TRUE)

map_data <- areas |>
  left_join(district_counts, by = c("district_id", "district_name", "area_km2"))
bb <- st_bbox(areas)
caption_text <- paste0(
  "Sources: City of Chicago Community Areas boundaries; Chicago library facilities (Problem 2). ",
  "CRS: EPSG:26916. Date: 2026-11-08. Sites unassigned (not exactly one match): ",
  nrow(unassigned_or_ambiguous), "."
)
base_theme <- theme_void() +
  theme(plot.title = element_text(face = "bold", size = 15),
        plot.caption = element_text(size = 8, hjust = 0.5),
        legend.position = "right")
```

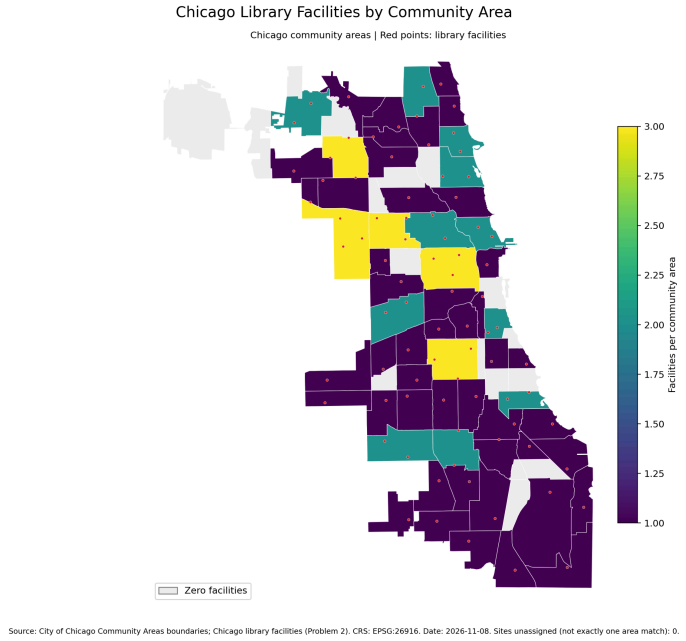
```
p_count <- ggplot() +
  geom_sf(data = filter(map_data, n_sites == 0),
          fill = "grey88", color = "white", linewidth = 0.25) +
  geom_sf(data = filter(map_data, n_sites > 0),
          aes(fill = n_sites), color = "white", linewidth = 0.25) +
  geom_sf(data = sites, color = "red3", size = 0.7) +
  scale_fill_viridis_c(name = "Facilities\n(count)", breaks = pretty,
                       limits = c(1, max(district_counts$n_sites))) +
  coord_sf(xlim = c(bb["xmin"], bb["xmax"]),
           ylim = c(bb["ymin"], bb["ymax"]), expand = FALSE) +
  labs(title = "Chicago Library Facilities by Community Area",
       subtitle = "Red points = facilities; gray polygons = zero facilities",
       caption = caption_text) + base_theme
ggsave(file.path(out, "p_count.png"), p_count, width = 9, height = 9, dpi = 300)

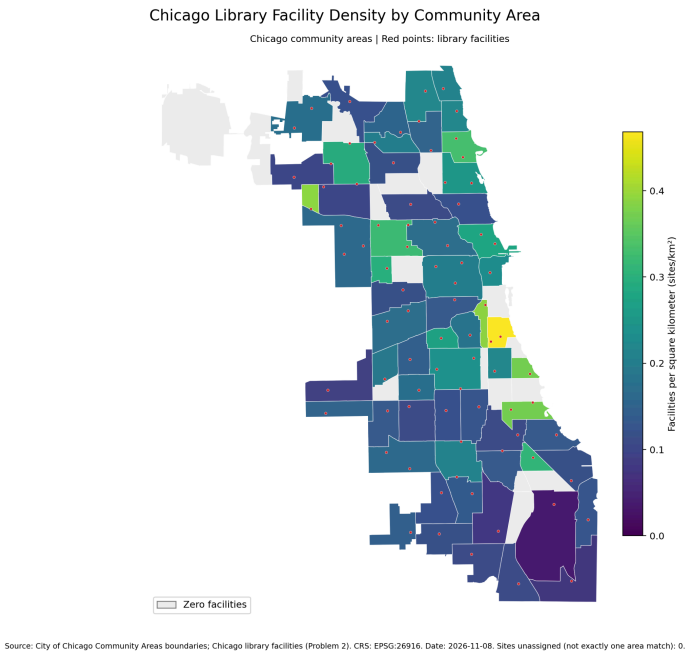
p_density <- ggplot() +
  geom_sf(data = filter(map_data, n_sites == 0),
          fill = "grey88", color = "white", linewidth = 0.25) +
  geom_sf(data = filter(map_data, n_sites > 0),
          aes(fill = sites_per_km2), color = "white", linewidth = 0.25) +
  geom_sf(data = sites, color = "red3", size = 0.7) +
  scale_fill_viridis_c(name = "Facilities per\nkm² (sites/km²)") +
  coord_sf(xlim = c(bb["xmin"], bb["xmax"]),
           ylim = c(bb["ymin"], bb["ymax"]), expand = FALSE) +
  labs(title = "Chicago Library Facility Density by Community Area",
       subtitle = "Red points = facilities; gray polygons = zero facilities",
       caption = caption_text) + base_theme
ggsave(file.path(out, "p_density.png"), p_density, width = 9, height = 9, dpi = 300)

data_sources <- tibble(
  dataset = c("Facilities", "Community area boundaries"),
  source = c("Cleaned facility locations from Problem 2; original uploaded Chicago Public Library
locations CSV",
             "City of Chicago Community Areas boundary dataset supplied for Problem 3"),
  file_or_description = c("cleaned_locations.csv; WGS84 coordinates",
                          "Boundaries_-_Community_Areas_20261108.csv; WGS84 MULTIPOLYGON
geometry"),
  access_date = c("2026-11-08", "2026-11-08")
)
write_csv(data_sources, file.path(out, "data_sources.csv"))

cat("\\nSite match totals (0, 1, multiple):\\n")
print(match_summary)
cat("\\nDistrict counts and density (all 77 areas):\\n")
```

```
print(district_counts, n = Inf)
cat("\\nHighest facility counts:\\n")
print(district_counts |> arrange(desc(n_sites), district_name) |> slice_head(n = 10))
cat("\\nHighest facility density:\\n")
print(district_counts |> arrange(desc(sites_per_km2), district_name) |> slice_head(n = 10))
```





The first map provides an overview of facility counts by area in Chicago while the second map takes into consideration distance in kilometers to map faculty density.